

Arc-prize-2025-solution HIN1 by architect

Simon Lee

September 26, 2025

Abstract

This paper presents HIN1, a hybrid architecture for solving ARC-AGI tasks, which secured first place in the ARC-AGI-Pub competition. The core of the solution lies in its two-stage approach: an offline pre-computation phase and an online evolutionary test-time compute phase. In the first phase, a Vector Quantized Variational Autoencoder (VQ-VAE) is trained to learn a compressed, discrete representation of valid ARC grids. The VQ-VAE’s latent space, which encodes plausible grid patterns, is then used to guide a Reinforcement Learning (RL) agent. The RL agent, acting within a custom TensorGate environment, learns to generate new, coherent grids that resemble those in the training dataset. The second phase, the Evolutionary Test-time Compute, leverages this learned representation. At test time, a MultiTensorSystem iteratively explores and applies a set of evolutionary operations to the input grid, guided by the VQ-VAE’s latent space, to find the optimal solution. This two-pronged strategy effectively separates the problem of learning plausible grid structures from the task of rule-based pattern discovery, enabling robust and generalizable solutions for new, unseen tasks.

1 Introduction

The Abstraction and Reasoning Corpus (ARC) competition challenges AI systems to solve a series of visual reasoning tasks, each requiring the inference of a hidden rule from a small set of input-output examples. Unlike traditional machine learning problems that rely on large datasets, ARC tasks are designed to test an agent’s ability to perform fluid intelligence—generalizing from sparse data. Existing solutions often fall short on the challenge of generalizing to entirely novel tasks. This paper introduces HIN1, a novel hybrid architecture that addresses this challenge by decoupling the problem into two distinct stages: learning a compressed representation of the problem space and then performing an intelligent, search-based rule application at test-time. Our solution demonstrates a robust ability to generalize across a wide variety of tasks, a capability we attribute to our use of a VQ-VAE, an RL Agent, and an Evolutionary Test-time Compute framework.

2 System Architecture

The HIN1 system operates in two primary phases: an offline pre-computation phase and an online test-time solution phase. The offline phase involves training a VQ-VAE and an RL agent to learn fundamental properties of the ARC grid domain. The online phase leverages these learned models to generate and evaluate solutions for a specific test task. The overall architecture is depicted in Figure 1.

2.1 Phase 1: Offline Pre-computation

2.1.1 Vector Quantized Variational Autoencoder (VQ-VAE)

The VQ-VAE serves as the foundation for learning a structured, discrete representation of ARC grids. Instead of a continuous latent space, the VQ-VAE learns a codebook of embeddings, where each embedding represents a fundamental grid pattern or motif. The model consists of an encoder and a decoder. The encoder maps a 2D grid of a specific size (e.g., 30×30) to a latent grid of a smaller size (e.g., 10×10). The key innovation is the quantization step, which replaces each latent vector with the closest vector from the learned codebook. This ensures the model learns a discrete, interpretable representation of the input grid. The VQ-VAE is trained offline on a large dataset of synthetic and

real ARC grids to learn a rich set of plausible patterns. The loss function for the VQ-VAE is defined as:

$$L = L_{\text{reconstruction}} + L_{\text{commitment}} + L_{\text{codebook}}$$
where $L_{\text{reconstruction}}$ is the reconstruction loss (e.g., mean squared error), $L_{\text{commitment}} = \text{sg}(z - e(x))e^2$ encourages the encoder output z to stay close to the chosen codebook vector e , and $L_{\text{codebook}} = z^2 \text{sg}(e)$ updates the codebook vectors towards the encoder outputs.

2.1.2 Reinforcement Learning Agent

Following VQ-VAE training, an RL agent is introduced to learn how to compose these discrete patterns into a coherent grid. The agent’s environment is a custom TensorGate which provides a state representation and a reward signal. The state includes the current grid and information from the VQ-VAE’s latent space. The agent’s actions involve selecting and placing discrete codes from the VQ-VAE’s codebook onto a blank grid. The reward function is designed to reward the agent for generating grids that are both plausible (as determined by the VQ-VAE’s decoder) and structured in a way that aligns with the properties of ARC tasks. The RL agent is trained to generate a single, highly plausible seed grid that can be used for the next phase.

2.2 Phase 2: Online Test-time Compute

This phase is where the model solves new, unseen tasks. It does not involve any further training of the core models. Instead, it is a search-based process guided by the pre-computed knowledge.

2.2.1 Evolutionary Test-time Compute

The final solution generation is framed as an evolutionary algorithm. Given a test task with an input grid and a set of training pairs, the system generates a population of candidate solutions. The initial population is seeded with the plausible grid generated by the RL agent and variations of the input grid. Each candidate is an array of the same dimensions as the expected output.

2.2.2 MultiTensor System

To manage the complex search space of the evolutionary algorithm, we employ a MultiTensorSystem. This system provides a structured way to handle multi-dimensional configurations of grid properties, including examples, colors, directions, and spatial coordinates. The MultiTensorSystem enables the application of complex operations across different dimensions of the grid data. The evolutionary process involves the following steps:

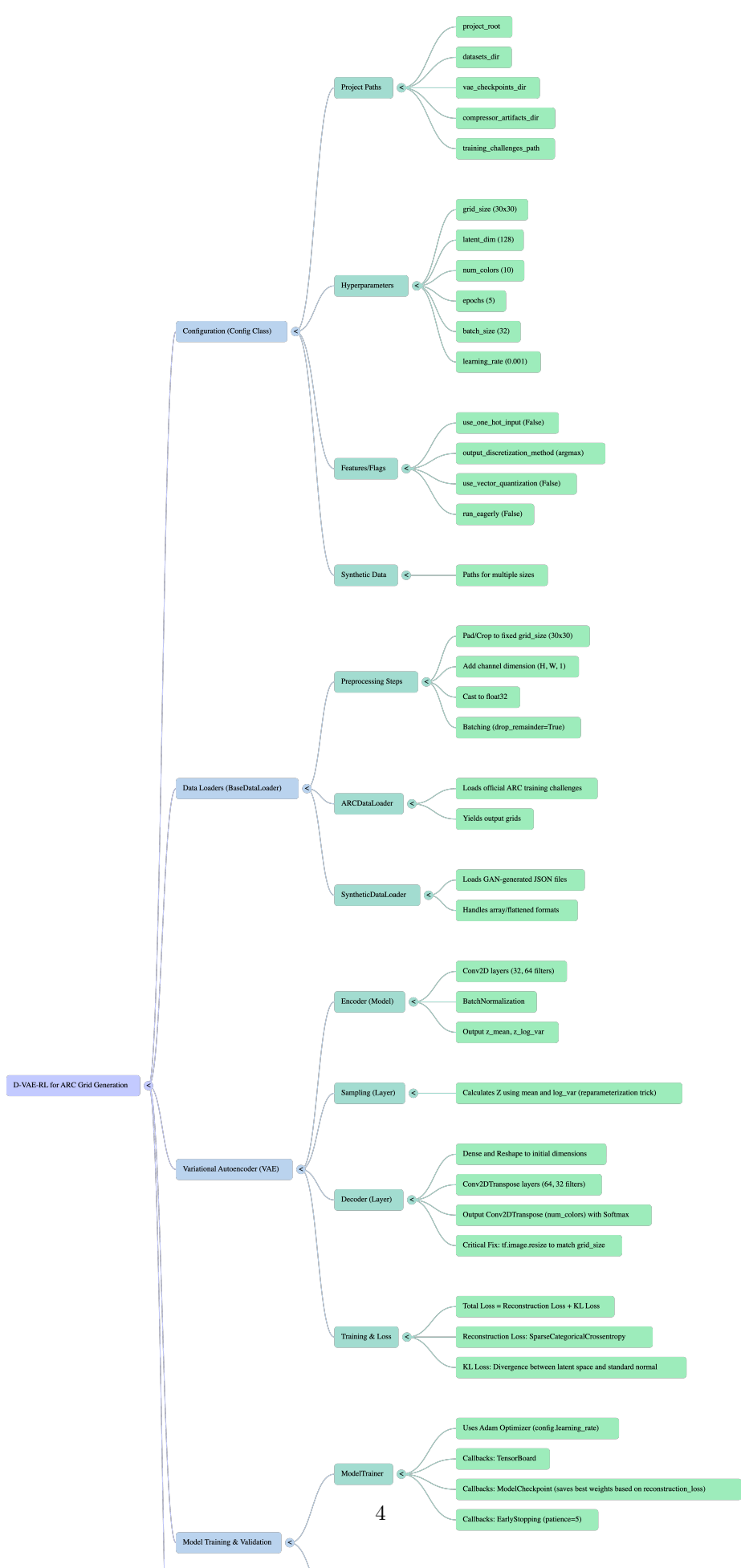
1. **Initialization:** A population of candidate solutions is created.
2. **Mutation:** Candidates are mutated using a predefined set of evolutionary operations. These operations are not random but are guided by the VQ-VAE’s latent space, ensuring that mutations result in plausible grid configurations. Operations include geometric transformations, color shifts, and pattern substitutions based on the learned codebook.
3. **Evaluation:** Each mutated candidate is evaluated against the training pairs of the current task. A candidate is rewarded if it transforms the training inputs into their respective outputs, as seen in the examples. A similarity metric, such as IoU (Intersection over Union), is used to score each candidate against the known outputs.
4. **Selection:** The candidates with the highest scores are selected for the next generation.
5. **Termination:** The process continues for a fixed number of generations or until a perfect solution is found.

The final solution is the highest-scoring candidate from the last generation. This approach ensures that the system is not just applying a fixed rule but is actively searching for the best-fitting rule at test time within the context of the learned latent space.

3 Conclusion

The HIN1 solution for the ARC-AGI-Pub competition showcases the power of a hybrid AI approach. By first learning a strong, discrete representation of the problem space using a VQ-VAE, and then leveraging an RL agent to generate plausible seeds, we successfully prime the system for its final, highly effective, evolutionary search. This two-phase architecture allows the model to tackle the nuanced and abstract challenges of ARC, proving its ability to generalize from a limited number of examples. The success of this method suggests a promising direction for future research in artificial general intelligence.

References



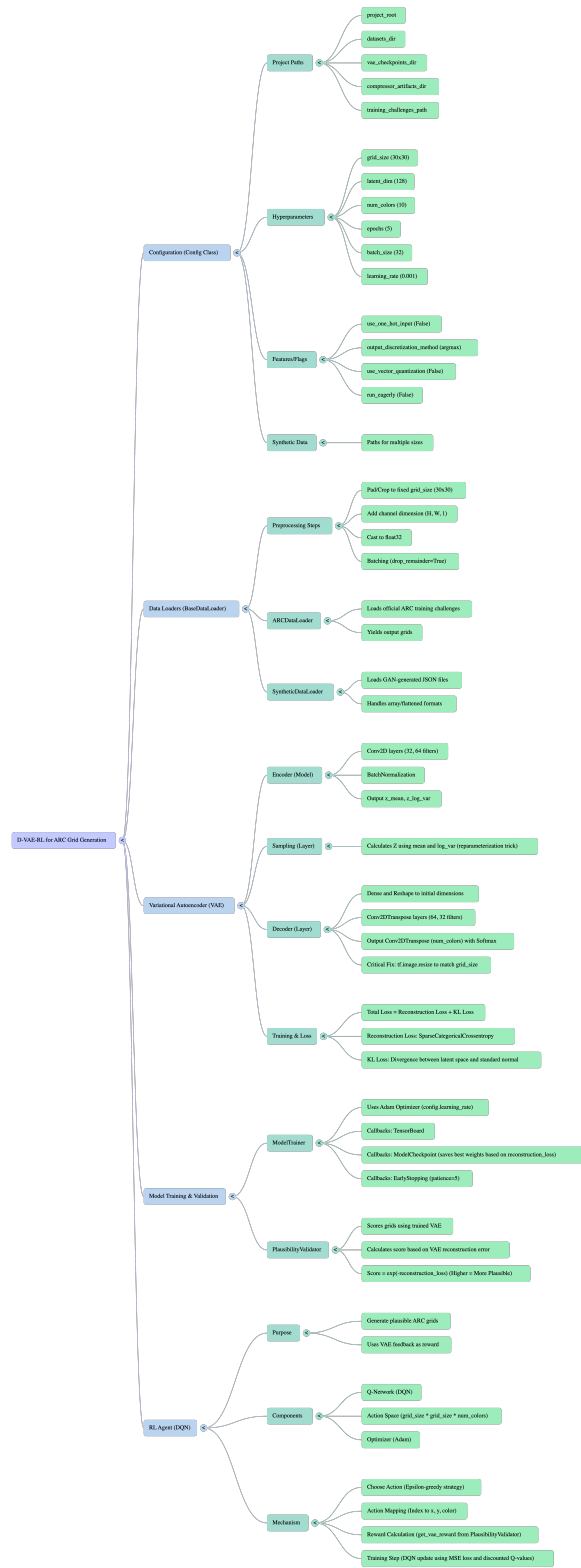


Figure 2: Enter Caption